

Estruturas de Dados - ESP412

Prof^a Ana Carolina Sokolonski

Bacharelado em Sistemas de Informação
Instituto Federal de Ciência e Tecnologia da Bahia
Campus de Feira de Santana

carolsoko@ifba.edu.br

February 13, 2025

Listas, Filas e Pilhas

1 Filas

2 Pilhas

3 Referências

Filas

Filas

A Fila consiste numa Estrutura de Dados que funciona obedecendo a ordem FIFO (First In First Out), ou seja, o primeiro a entrar é o primeiro a sair. A Fila pode ser implementada com vetores ou com listas, ou seja com ou sem ponteiros, contanto que obedeça a regra FIFO. Aqui na disciplina usaremos ponteiros em C para implementar nossa Fila.



Filas

As filas são muito utilizadas na computação em diversos campos, como filas de processos em Sistemas Operacionais, filas de pacotes em Redes de Computadores, entre outros exemplos. Então, nenhum profissional de computação deve se eximir de aprender com maestria como manipular uma fila. Na verdade, nenhuma estrutura de dados!



Filas

Basicamente, quando pensamos em filas, pensamos em duas ações importantes: **Enfileira** e **Desenfileira**, ou seja, o elemento chegou e entrará no fundo da fila (enfileira) e o elemento já esperou na fila e agora está no topo e chegou a hora de ser atendido (desenfileira).



Filas

Struct:

A *STRUCT* de uma fila pode ser construída de forma idêntica a uma lista duplamente encadeada.

Filas

Struct:

A *STRUCT* de uma fila pode ser construída de forma idêntica a uma lista duplamente encadeada.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct reg{
    int          conteudo;
    struct reg   *prox;
    struct reg   *ant;
}celula;
```


Filas

Enfileirar:

A função **Enfileirar**, que corresponde a função “inserir” na fila, deve ser realizada de forma que o elemento inserido seja colocado sempre no “FINAL” da fila.

Filas

Enfileirar:

A função **Enfileirar**, que corresponde a função “inserir” na fila, deve ser realizada de forma que o elemento inserido seja colocado sempre no “FINAL” da fila.

```
celula *enfileirar(int x, celula *fila){
    celula *novo, *p;
    p = fila;
    novo = malloc (sizeof (celula));
    novo->conteudo = x;
    novo->prox = NULL;
    novo->ant = NULL;
    if (p!=NULL){
        for(p;p->prox!= NULL;p=p->prox);
        p->prox=novo;
        novo->ant=p;
    }else fila = novo;
    return fila;
}
```

Filas

Desenfileirar:

A função **Desenfileirar**, que corresponde a função “remover” da fila, deve ser realizada de forma que o elemento removido seja removido sempre do “TOPO” da fila.

Filas

Desenfileirar:

A função **Desenfileirar**, que corresponde a função “remove” da fila, deve ser realizada de forma que o elemento removido seja removido sempre do “TOPO” da fila.

```
celula *desenfileirar(celula *fila){
    celula *p, *q;
    if (fila!=NULL){
        q = fila;
        p=q->prox;
        p->ant = NULL;
        free(q);
        return p;
    }else return fila;
}
```

Filas

Buscar:

A função buscar na fila percorre a fila e retorna um ponteiro para o elemento buscado.

Filas

Buscar:

A função buscar na fila percorre a fila e retorna um ponteiro para o elemento buscado.

```
celula *buscar(int x, celula *fila){
    celula *aux = fila;
    while(aux!=NULL && aux->conteudo!=x)
        aux=aux->prox;
    return aux;
}

celula *buscarRecurso(int x, celula *fila){
    if (fila == NULL) return NULL;
    if (fila->conteudo!=x) return buscarRecurso(x,fila->prox);
    else return fila;
}
```

Filas

Imprimir:

A função imprimir mostra todos os elementos da fila na ordem do último para o primeiro, como mostrado na figura abaixo.

Filas

Imprimir:

A função imprimir mostra todos os elementos da fila na ordem do último para o primeiro, como mostrado na figura abaixo.



Filas

Imprimir:

A função imprimir mostra todos os elementos da fila na ordem do último para o primeiro, como mostrado na figura abaixo.



```
void imprimir(celula *fila) {  
    celula *p;  
    for (p = fila; p != NULL; p = p->prox){  
        printf ("%d\n", p->conteudo);  
    }  
}
```

Filas

```
int main(){
    int resp;
    printf("Fila\n");
    celula *fila, *p;

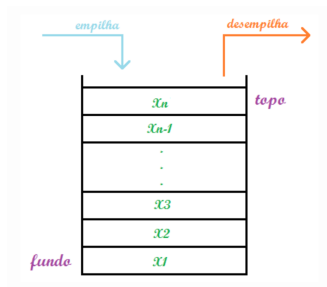
    for(int j=0; j<10;j++){
        printf("Digite o próximo elemento da fila: ");
        scanf("%d", &resp);
        fila = enfileirar(resp, fila);
        imprimir(fila);
    }
    printf("Fim do enfileiramento \n");

    desenfileirar(fila);
    desenfileirar(fila);
    imprimir(fila);
    printf("desenfileirou dois elementos \n");
    return 0;
}
```

Pilhas

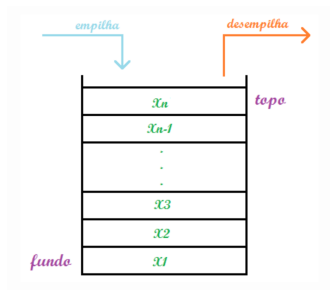
Pilhas

Pilha é uma estrutura de dados que obedece a regra em que os elementos são removidos na ordem do inversa daquela em que foram inseridos de modo que o último elemento que entra é sempre o primeiro a ser retirado, por isto este tipo de estrutura de dados é chamada LIFO (Last In - First Out) ou FILO (First In - Last Out).



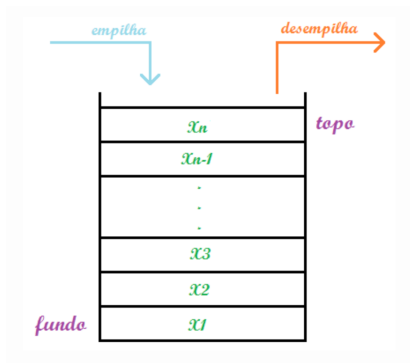
Pilhas

O exemplo mais prático para se entender uma pilha é uma pilha de livros ou pilha de pratos, no qual ao se colocar diversos elementos uns sobre os outros, se quisermos pegar o livro ou o prato mais abaixo deve-se tirar todos os livros, ou pratos, que estão por cima. Outro exemplo é um elevador, a primeira pessoa a entrar num elevador, normalmente, é a última pessoa a sair do elevador.



Pilhas

Basicamente, quando pensamos em pilhas, pensamos em duas ações importantes: **Empilha** e **Desempilha**, ou seja, o elemento chegou e entrará em cima da pilha (empilha) e o elemento será atendido e sairá da pilha (desempilha).



Pilhas

Struct, Imprimir e Buscar:

A *STRUCT*, a função imprimir e a função buscar da Pilha são idênticas às funções da Fila, então não repetiremos elas aqui. Podemos consultar os Slides anteriores ou o Código fonte.

Pilhas

Empilhar:

A função **Empilhar**, que corresponde a função “inserir” na pilha, deve ser realizada de forma que o elemento inserido seja colocado sempre no “TOPO” da pilha.

Pilhas

Empilhar:

A função **Empilhar**, que corresponde a função “inserir” na pilha, deve ser realizada de forma que o elemento inserido seja colocado sempre no “TOPO” da pilha.

```
celula *empilhar(int x, celula *pilha){  
    celula *novo;  
    novo = malloc (sizeof (celula));  
    novo->conteudo = x;  
    novo->prox = NULL;  
    novo->ant = NULL;  
    if (pilha!=NULL){  
        novo->prox = pilha;  
        pilha->ant = novo;  
        pilha = novo;  
    }else pilha = novo;  
    return pilha;  
}
```

Pilhas

Desempilhar:

A função **Desempilhar**, que corresponde a função “remove” da pilha, deve ser realizada de forma que o elemento removido seja sempre retirado do “TOPO” da pilha.

Pilhas

Desempilhar:

A função **Desempilhar**, que corresponde a função “remove” da pilha, deve ser realizada de forma que o elemento removido seja sempre retirado do “TOPO” da pilha.

```
celula *desempilhar(celula *pilha){
    celula *p, *q;
    if (pilha!=NULL){
        q = pilha;
        p=q->prox;
        p->ant = NULL;
        free(q);
        return p;
    }else return pilha;
}
```

Pilhas

```
int main(){
    int resp;
    printf("Pilha\n");
    celula *pilha,*p;

    for(int j=0; j<10;j++){
        printf("Digite o próximo elemento da Pilha: ");
        scanf("%d", &resp);
        pilha = empilhar(resp, pilha);
        imprimir(pilha);
    }
    printf("Fim do empilhamento \n");

    pilha = desempilhar(pilha);
    pilha = desempilhar(pilha);
    pilha = desempilhar(pilha);
    imprimir(pilha);
    printf("Desempilhou três elementos \n");
    return 0;
}
```

Referências

Referências